



北京大学

硕士研究生学位论文

题目： 基于 XXXX 的 XXXX 系
统设计与实现

姓 名： 扎克·施耐德

学 号： 180XXXXXXXX

院 系： XXXXXX 学院

专 业： XXXX

研究方向： XXXX

导 师： XXX 教授

YYY 教授

二〇二二 年 六 月

版权声明

任何收存和保管本论文各种版本的单位和个人，未经本论文作者同意，不得将本论文转借他人，亦不得随意复制、抄录、拍照或以其他方式传播。否则，引起有碍作者著作权之问题，将可能承担法律责任。



摘要

中文摘要部分...

关键词：A，B，C，D

Design and implementation of a XXXXX system based on XXXX

Zack Snyder (XXXX)

Directed by: Prof. XXX and Prof. YYY

ABSTRACT

英文摘要部分...

KEY WORDS: A,B,C,D

目录

第一章	引论.....	1
第二章	理论基础与问题模型.....	3
第三章	两种基于动态延迟主元的波前矩阵分解 Kernel.....	5
3.1	非对称稠密矩阵的延迟主元 LU 方法.....	5
3.1.1	分解 Fully Summed 块.....	5
3.1.2	更新波前矩阵.....	8
3.1.3	传递贡献块给父节点.....	9
3.2	对称稠密矩阵的延迟 Bunch–Kaufman 方法.....	10
3.2.1	Bunch–Kaufman 选主元.....	11
3.2.2	块分解优化.....	13
3.3	两种稠密矩阵分解算法的 CUDA Kernel 实现	14
第四章	面向延迟主元的 GPU 管理方法	15
第五章	实验结果与分析	17
第六章	总结和展望	19
	攻读硕士期间发表的论文及其他成果	21
	致谢	23
	北京大学学位论文原创性声明和使用授权说明	25

表格索引

表 3.1 节点分区说明	13
--------------------	----

插图索引

图 3.1	非对称超节点对应的波前矩阵.....	5
图 3.2	非对称 Fully Summed 块的一步主元分解过程	6
图 3.3	Step 1 完成后波前矩阵的分区。.....	8
图 3.4	延迟主元贡献块向父节点传递.....	9
图 3.5	Bunch-Kaufman 选主元.....	11

主要符号对照表

x, y, m, n, t	标量, 通常为变量
K, L, D, M, N, T	标量, 通常为超参数
$x \in \mathbb{R}^D$	D 维列向量
$(x_1, \dots, x_D)$	D 维行向量
$(x_1, \dots, x_D)^T$ or $(x_1; \dots; x_D)^T$	D 维行向量
$\mathbf{A} \in \mathbb{R}^{K \times D}$	大小为 $K \times D$ 的矩阵
$x \in \mathbb{R}^{KD}$	(KD) 维的向量
$\mathbb{M}_i$ or $\mathbb{M}_i(\mathbf{x})$	第 i 列为 $\mathbf{1}$ (或者 $\mathbf{x}$), 其余为 $\mathbf{0}$ 的矩阵
$diag(\mathbf{x})$	对角矩阵, 其对角元素为 $\mathbf{x}$
$\mathbf{I}_N$ or $\mathbf{I}$	$(N \times N)$ 的单位阵
$diag(\mathbf{A})$	列向量, 其元素为 $\mathbf{A}$ 的对角元素
$\mathbf{A} \in \mathbb{R}^{D_1 \times D_2 \times \dots \times D_K}$	大小为 $D_1 \times D_2 \times \dots \times D_K$ 的张量
$\{x^{(n)}\}_{n=1}^N$	集合
$\{(x^{(n)}, y^{(n)})\}_{n=1}^N$	数据集
$\mathcal{N}(\mathbf{x}; \mu, \Sigma)$	变量 x 服从均值为 μ , 方差为 Σ 的高斯分布

① 本符号对照表内容选自qiu2020nndl老师的《神经网络与深度学习》qiu2020nndl一书。

第一章 引论

第二章 理论基础与问题模型

针对常见的稠密矩阵,我们在求解时往往采用

第三章 两种基于动态延迟主元的波前矩阵分解 Kernel

在超节点方法中，需要把单个超节点的主元块作为稠密矩阵进行 LU 或 $L(D)L^T$ 分解。按照列主元方法，算法逐列选择主元并进行更新。如果某一列待分解的部分全为零，就无法选出列主元。LAPACK 的 `dgetrf` 函数在大量的求解器中担负这个责任，但当其遇到上述问题的时候则会跳过该零列，继续分解后续列并在最后报告错误；这种处理不能满足我们对不正定但是满秩可分解矩阵的分解流程的需要。

一个子主元块无法分解，并不代表整个矩阵无法分解。当前节点不能消去的部分可以传递给父节点，再在父节点中尝试分解。

需要处理的波前矩阵由四部分组成：

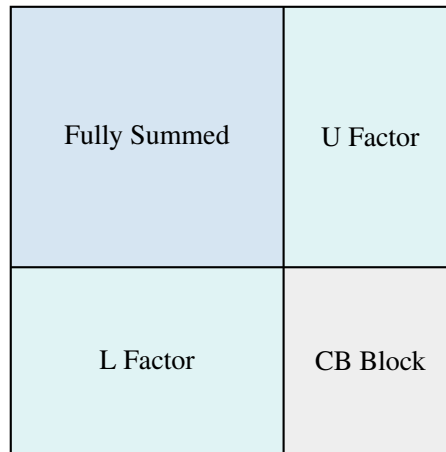


图 3.1 非对称超节点对应的波前矩阵

- Fully Summed 是已经装配完成、可以进行分解的矩阵，即超节点的对角块；
- L/U Factor 是需要根据 Fully Summed 的分解结果更新的块，最终作为因子保存；
- CB Block 是子节点传给父节点的贡献块，需要在更新后传递给父节点；
- 对角块之外的行列来自与对角块中的行列相耦合的变量。

我们针对上述问题提出了两种基于动态延迟主元的波前矩阵分解 Kernel，分别用于处理对称和非对称稠密矩阵。

3.1 非对称稠密矩阵的延迟主元 LU 方法

3.1.1 分解 Fully Summed 块

首先单独考虑 Fully Summed 块的分解。已经分解更新的部分包含 L_{EE} 、 U_{EE} 、 L_{DE} 和 U_{ED} ，接下来需要继续分解 Unfactored Block。一个朴素的想法是：首先选择列主元

(即该列待更新的行中最大元素)，如果能够选到合适的列主元，则进行行交换 P ，并用该元素作为主元；如果不能选到合适的列主元，则在整个活动子块中选择主元（即待更新的子块中最大的元素），此时需要同时进行行、列交换，将活动子块的全局最大元素置换到当前主元位置，根据该主元计算 L_p ，再更新 U_p 和右下角子块。

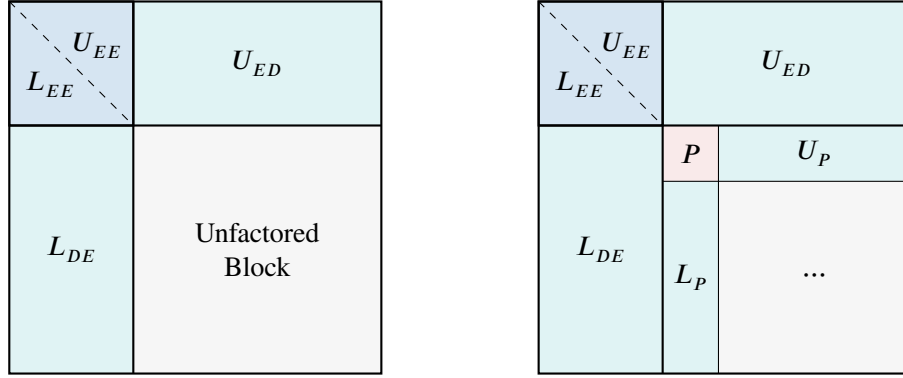


图 3.2 非对称 Fully Summed 块的一步主元分解过程

3.1.1.1 主元接受阈值

设第 k 步尚未分解的子块为 $S^{(k)} = [s_{ij}^{(k)}]_{i,j=k}^f$ ，其中 f 为 Fully Summed 块的阶数， $\tau \in (0, 1)$ 为主元接受阈值。列主元只搜索活动子块第 k 列，候选列主元所在行为：

$$p_k = \operatorname{argmax}_{k \leq i \leq f} |s_{ik}^{(k)}|, \quad \alpha_k = |s_{p_k k}^{(k)}|.$$

记当前子块的候选主元为 h_k ：当 $h_k \geq \tau \alpha_k$ 时，认为当前的候选主元主元足够稳定，于是不用做交换就可以利用其作为主元进行分解。反之 $h_k < \tau \alpha_k$ 时，说明需要将候选列主元选作主元，即将当前行与第 P 行作交换。

但是还存在一种特殊情况：当前列为全零列，选出的候选主元和候选列主元都为零，就无法选出列主元。此时需要在整个活动子块中搜索最大元素作为主元。记活动子块中的最大元素为：

$$\beta_k = \|S^{(k)}\|_{\max} = \max_{k \leq i, j \leq f} |s_{ij}^{(k)}|, \quad (p_k, q_k) = \operatorname{argmax}_{k \leq i, j \leq f} |s_{ij}^{(k)}|.$$

分别用 P_k 和 Q_k 将第 p_k 行、第 q_k 列交换到第 k 行、第 k 列：

$$\tilde{S}^{(k)} = P_k S^{(k)} Q_k, \quad d_k = \tilde{s}_{kk}^{(k)} = s_{p_k q_k}^{(k)}, \quad |d_k| = \beta_k.$$

因此，只使用列主元时只需要记录行置换，最终分解满足 $PA = LU$ ；如果使用过全主元，则必须记录行、列两种置换，最终满足 $PAQ = LU$ 。

3.1.1.2 列延迟优化

直接进行全主元搜索和逐项更新会消耗大量时间，这样的性能差距是我们无法接受的，所以我们提出了列延迟优化方法来取代全主元搜索。

全主元通过行列变换把全局最大元素移动到当前主元位置，本质上是先把全局最大元素所在列交换到第 k 列，再在换入列中进行一次列主元分解，相当于提前对这一列进行了选主元分解的步骤。于是我们可以反其道而行之，在原来的第 k 列在活动部分全为零时，可以直接移到未分解区域尾部。被移动到右侧的列在当前活动部分满足：

$$A(k : m - 1, j) = 0.$$

而后续 LU 更新为：

$$A(k + 1 : m - 1, j) \leftarrow A(k + 1 : m - 1, j) - L(:, k)U(k, j).$$

由于 $U(k, j) = 0$ ，更新项也为零，该列在后续分解中仍保持为零。因此，全主元步骤可以替换为以下列延迟步骤：

1. 若 $\alpha_k = 0$ ，将该列移动到 Unfactored Block 的尾部，并记录此次列交换；
2. 从右向左寻找第一列可用列（为防止已进入尾部的零列被多次交换，搜索范围必须排除已形成的延迟列），找到后立即停止，不再比较其他列；
3. 在该列内部使用列主元法，并将该列与当前被延迟的列交换；
4. 如果搜索到当前活动区的起点仍找不到可用列，说明整个 Unfactored Block 已经全零，判定为将剩余主元全部延迟。

3.1.1.3 right-looking 更新

主元 d_k 确定后，对主元行、主元列和右下角剩余子块执行 right-looking 更新：

$$(L_p)_i = \frac{\tilde{s}_{ik}^{(k)}}{d_k}, \quad k < i \leq f, \quad (U_p)_j = \tilde{s}_{kj}^{(k)}, \quad k < j \leq f,$$

$$s_{ij}^{(k+1)} = \tilde{s}_{ij}^{(k)} - (L_p)_i (U_p)_j = \tilde{s}_{ij}^{(k)} - \frac{\tilde{s}_{ik}^{(k)} \tilde{s}_{kj}^{(k)}}{d_k}, \quad k < i, j \leq f.$$

若 $\beta_k = 0$ ，则 $S^{(k)}$ 是全零块，列主元和全主元均不存在，在数值意义下不能继续分解，需要将未分解变量延迟到父节点。

完成上述步骤后， P 、 U_p 和 L_p 归入已分解部分，剩余部分继续作为 Unfactored Block 进行下一步迭代。

3.1.1.4 自适应 KJI-SAXPY 分块优化

这里可以引用下那篇论文的 **KJI-SAXPY** 方法

如果每次选择主元后立即更新所有剩余元素，计算会包含大量细粒度操作。受到 LAPACK 的 `dgetrf` 函数的启发，我们选择借助 cuBLAS 三角求解和矩阵乘法对矩阵块进行集中更新，即先分解一部分主元，再统一更新剩余部分。

示例选择的 panel 长度为 64。在 panel 内选择列主元时，搜索范围包含当前 64 列的全部活动行的长方形矩阵。选择主元后，只更新当前长方形 panel 中的数值；只要保证下一次选择主元之前当前列已经包含此前主元的更新即可。完成一个 panel 后，再更新其余部分并开始新的 panel。

延迟主元会影响正在处理的 panel。为了保证数值分解正确，算法在遇到延迟主元时立即停止当前 panel 的分解，根据当前 panel 已经接受的主元把矩阵剩余部分完整更新一次，然后从交换进来的非零列开始新的 panel。这样既使用 cuBLAS 加速了解，又保证主元搜索作用于最新的 Schur 补。

测试中，列延迟方法使整个计算过程的时间缩短约 90%；采用更细粒度的自适应 KJI-SAXPY 方法后，速度还能提升约一倍。整体优化后的算子在经过特别设计的算例上的运行时间已经略快于 LAPACK。

3.1.2 更新波前矩阵

Fully Summed 部分完成分解后，波前矩阵按已分解变量 E 、延迟变量 D 和普通更新变量 U 分成以下部分：

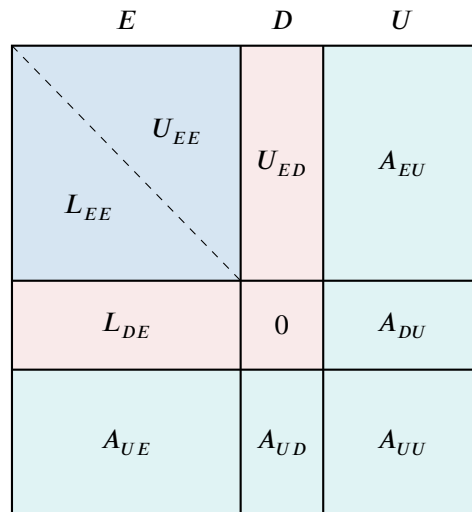


图 3.3 Step 1 完成后波前矩阵的分区。

数学推导？

首先更新 A_{EU} 和 A_{UE} 为 U_{EU} 和 L_{UE} ，利用三角求解计算：

$$U_{EU} = L_{EE}^{-1} A_{EU} \quad L_{UE} = A_{UE} U_{EE}^{-1}.$$

其次需要更新： A_{DU} 、 A_{UD} 和 A_{UU} ，分别计算：

$$A_{DU} \leftarrow A_{DU} - L_{DE} U_{EU} \quad A_{UD} \leftarrow A_{UD} - L_{UE} U_{ED} \quad A_{UU} \leftarrow A_{UU} - L_{UE} U_{EU}.$$

3.1.3 传递贡献块给父节点

子节点的波前矩阵更新完成后，将右下的四个子矩阵组成扩展贡献块，并传递给父节点装配。装配过程需根据贡献块的全局行、列编号执行 extend-add，普通更新变量与父节点已有索引匹配后进行累加；子节点未消去的延迟变量则需要动态扩充父节点的候选主元区域，将其规划进父节点的 Fully Summed 块中。

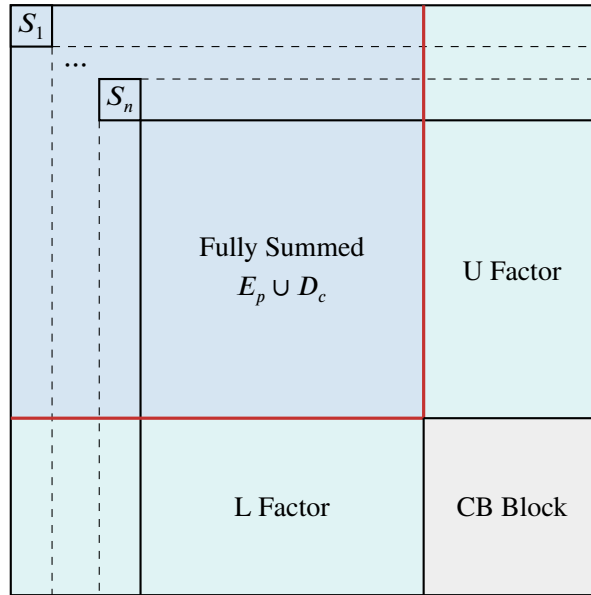


图 3.4 延迟主元贡献块向父节点传递

设子节点已消去的变量为 E_c ，未能消去的延迟变量为 D_c ，原来的普通更新变量为 U_c 。则子节点传出的扩展贡献块为：

$$C_c = \begin{bmatrix} S_{DD} & S_{DU} \\ S_{UD} & S_{UU} \end{bmatrix}, \quad I_c^{\text{out}} = D_c \cup U_c.$$

除了数值 C_c ，还必须同时传递其全局行、列索引数组。对于非对称 LU，行索引和列索引应分别保存： $R_c^{\text{out}} = D_c^r \cup U_c^r$ 、 $C_c^{\text{out}} = D_c^c \cup U_c^c$ 。

设父节点原来的候选主元集为 E_p ，待更新集为 U_p 。正确的符号分析通常保证了子

节点的普通更新变量已经位于父波前中，即：

$$U_c \subseteq E_p \cup U_p.$$

子节点的普通更新变量到达父节点后，需要根据符号分析确定的消去归属重新分类：

- $U_c \cap E_p$ 在子节点中只是更新变量，但到达父节点后已经 Fully Summed，可以在父节点中作为候选部分参与主元选择；
- $U_c \cap U_p$ 仍然位于父节点的待更新区。

延迟变量 D_c 是数值分解过程中动态产生的，父节点需要将其加入新的候选主元集：

$$E_p^{\text{new}} = E_p \cup D_c, \quad I_p^{\text{new}} = E_p^{\text{new}} \cup U_p.$$

得到了子节点传过来贡献块数据后，父节点为每个全局行、列编号建立到局部存储位置的映射：

$$\text{map}_r(g) = g \text{ 在父波前行索引中的局部位置.}$$

$$\text{map}_c(g) = g \text{ 在父波前列索引中的局部位置.}$$

如果扩展后出现新的行或列，先扩展父波前的存储空间，并把新位置初始化为零。然后再将所有子贡献块中全局编号一致的位置累加到父波前的新位置上：

$$F_p(\text{map}_r(R_c^{\text{out}}(i)), \text{map}_c(C_c^{\text{out}}(j))) += C_c(i, j).$$

父矩阵中原来没有数值的位置按零处理，累加子节点贡献后成为新的填充。如果多个子节点对同一全局位置都有贡献，必须把这些贡献全部累加，不能相互覆盖。

完成所有子贡献块的 extend-add 后，父节点按照以下规则处理：

- $E_p \cup D_c$ 放入 Fully Summed 候选主元区，在父节点重新执行主元检验；
- 在父节点重复上述的选主元和更新步骤，成功消去的延迟变量进入 L 、 U 因子；
- 仍无法通过主元检验的变量再次进入父节点的贡献块，继续向更高层节点传递。

3.2 对称稠密矩阵的延迟 Bunch–Kaufman 方法

如果整个波前矩阵是对称的，则 Fully Summed 块也是对称的。此时采用 Bunch–Kaufman 方法进行 LDL^T 分解：

$$P^T A P = L D L^T.$$

其中， P 是同时置换行、列的置换矩阵， L 是单位下三角矩阵， D 是由 1×1 和 2×2 主元块组成的块对角矩阵。与非对称 LU 不同，对称分解不能只交换行而不交换列；交换

第 i 、 j 行时，必须同时交换第 i 、 j 列。

所以我们在对对称矩阵处理时主元块分解的方法和非对称矩阵不同，而剩余的更新步骤和非对称情况保持一致。

3.2.1 Bunch–Kaufman 选主元

设第 k 步尚未分解的活动对称子块为 $S^{(k)} = [s_{ij}^{(k)}]_{i,j=k}^f$ ，其中 f 为 Fully Summed 块的阶数。Bunch–Kaufman 方法在每一步选择一个 1×1 或 2×2 对角主元块，并保持后续更新仍然对称。定义 Bunch–Kaufman 阈值如下：

$$\gamma = \frac{1 + \sqrt{17}}{8} \approx 0.640388.$$

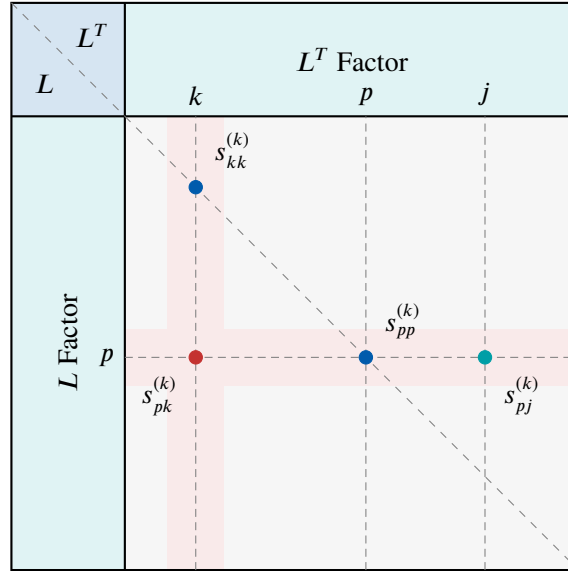


图 3.5 Bunch–Kaufman 选主元

首先检查第 k 列，记当前对角元大小为 $a_k = |s_{kk}^{(k)}|$ ，其对角线以下最大元素为：

$$\lambda_k = \max_{k < i \leq f} |s_{ik}^{(k)}| = |s_{pk}^{(k)}|, \quad p = \operatorname{argmax}_{k < i \leq f} |s_{ik}^{(k)}|.$$

如果该列不全零，则按照以下规则选择主元：

1. 若 $a_k \geq \gamma \lambda_k$ ，直接采用 $s_{kk}^{(k)}$ 作为 1×1 主元，不进行交换。
2. 若 $a_k < \gamma \lambda_k$ ，则检查候选行 p 。定义该行除对角元素之外的最大元素：

$$\sigma_p = \max_{\substack{k \leq j \leq f \\ j \neq p}} |s_{pj}^{(k)}|$$

因为 $|s_{pk}^{(k)}| = \lambda_k$, 所以 $\lambda_k > 0$ 时有 $\sigma_p \geq \lambda_k > 0$ 。随后依次判断:

- (a) 若 $a_k \geq \gamma \frac{\lambda_k^2}{\sigma_p}$, 仍然使用 $s_{kk}^{(k)}$ 作为 1×1 主元;
- (b) 否则, 若 $|s_{pp}^{(k)}| \geq \gamma \sigma_p$, 同时交换第 k 、 p 行和第 k 、 p 列, 把 $s_{pp}^{(k)}$ 移到 (k, k) , 并将其作为 1×1 主元;
- (c) 若以上两个条件均不满足, 选择由第 k 、 p 个变量组成的 2×2 主元块。通过对称交换将变量 p 移动到 $k+1$ 位置, 得到:

$$D_k = \begin{bmatrix} \tilde{s}_{kk}^{(k)} & \tilde{s}_{k,k+1}^{(k)} \\ \tilde{s}_{k+1,k}^{(k)} & \tilde{s}_{k+1,k+1}^{(k)} \end{bmatrix}.$$

对于 1×1 主元 $s_{pp}^{(k)}$, 用置换矩阵 P_k 同时交换第 k 、 p 行和第 k 、 p 列:

$$\tilde{S}^{(k)} = P_k^T S^{(k)} P_k, \quad D_k = [\tilde{s}_{kk}^{(k)}].$$

对于由 k 、 p 组成的 2×2 主元, 用 P_k 将这两个变量置换到活动子块的前两个位置:

$$\tilde{S}^{(k)} = P_k^T S^{(k)} P_k = \begin{bmatrix} D_k & B_k^T \\ B_k & C_k \end{bmatrix}, \quad D_k \in \mathbb{R}^{2 \times 2}.$$

置换必须同时作用于整个波前矩阵的相应行和列, 并同步更新全局变量编号。多步置换累积后, 最终得到:

$$P = P_1 P_2 \cdots P_l, \quad P^T F P = L D L^T.$$

完成对称置换后, 当前块列的消元乘子为:

$$L_k = B_k D_k^{-1}.$$

右下角活动子块通过对称 Schur 补更新:

$$S^{(k+r)} = C_k - L_k D_k L_k^T = C_k - B_k D_k^{-1} B_k^T.$$

对于 1×1 主元 $D_k = [d_k]$, 公式退化为:

$$L_k = \frac{B_k}{d_k}, \quad S^{(k+1)} = C_k - \frac{B_k B_k^T}{d_k}.$$

对于 2×2 主元, 更新为:

$$L_k = \frac{1}{\Delta_k} B_k \begin{bmatrix} s_{pp}^{(k)} & -s_{pk}^{(k)} \\ -s_{pk}^{(k)} & s_{kk}^{(k)} \end{bmatrix}, \quad S^{(k+2)} = C_k - L_k D_k L_k^T.$$

这里就可以看出选择 2×2 主元的精妙之处: 因为整个过程涉及到需要求矩阵的逆, 因

此矩阵行列式的数值就关乎数值稳定性的结果：如果其行列式值过小，则会导致数值溢出等的问题。

观察 2×2 主元的行列式值为 $\Delta_k = s_{kk}^{(k)} s_{kk}^{(k)} - s_{pk}^{(k)} s_{pk}^{(k)}$ ，注意到出现 2×2 主元说明前序 1×1 判定已经全部失败，于是就有：

$$|s_{kk}^{(k)}| < \frac{\gamma}{\sigma_p} (s_{pk}^{(k)})^2, \quad |s_{pp}^{(k)}| < \gamma \sigma_p.$$

带入到行列式的计算中，得到： $|\Delta_k| < (1 - \gamma^2) (s_{pk}^{(k)})^2$ ，由于阈值 γ 已经选定，所以当列主元不为极小的值时，行列式的绝对值不会过小，保证了数值稳定性。

但是对于不正定的矩阵仍然会存在找不到列主元的情况：即当前列全零，这样的情况下该选主元策略也会失效。为了处理这种情况，我们类似于非对称的处理将其移动到最后一个主元位置，准备将其延迟到父节点的波前矩阵上再做处理。

3.2.2 块分解优化

与非对称情况相同，对称算法也使用分块方法优化。算法维护四个连续区间：

表 3.1 节点分区说明

区间	说明
$[0, \text{panel_begin})$	以前面板已经消去
$[\text{panel_begin}, k)$	当前面板已经接受的主元
$[k, \text{active_end})$	仍可参与主元选择的活跃节点
$[\text{active_end}, n)$	已经延迟的节点

设当前 panel 从 $b = \text{panel_begin}$ 开始。以前 panel 产生的 Schur 补已经全部写回 A ，所以 panel 开始时 $A(b : n - 1, b : n - 1)$ 就是最新的尾矩阵。

假设当前 panel 已经接受了 t 列，令 $L_P = L(:, b : b + t - 1)$ ，相应的 1×1 和 2×2 主元组成 D_P 。定义工作矩阵 $W_P = L_P D_P$ ，当前真正的 Schur 补为：

$$S = A_{\text{stored}} - L_P D_P L_P^T = A_{\text{stored}} - L_P W_P^T.$$

准备处理第 k 列时， $A(:, k)$ 尚未包含当前 panel 前面 t 列主元的更新，因此必须先计算当前列的更新，再继续选择主元。

按照 Bunch–Kaufman 方法，首先判断当前列的对角元素是否满足阈值要求；如果满足，则把它标记为主元。如果不满足，后续判定需要使用当前列最大元素所在行的数据，因此还需要更新该行。当 1×1 、 2×2 主元均不满足要求时，需要延迟主元。算法维护 `active_end`，记录尾部延迟行、列的边界；每次延迟时，从该边界向左寻找下一个非零列，再将其与被延迟主元交换。

每个 panel 的分解过程维护工作数组 W_P 。理论上可以只保存 L_P, D_P ，但每次需要当前列时，都必须先计算 $D_P L_P(k, :)^T$ ，再计算 $L_P (D_P L_P(k, :)^T)$ 。

由于 D_P 混合了 1×1 和 2×2 块，反复执行这一过程较为复杂。预先保存 $W_P = L_P D_P$ 后，当前列可直接计算 $L_P W_P(k, :)^T$ ，只需要一次 `dgemv`，不必在每次更新时重新遍历 D_P 的 $1 \times 1/2 \times 2$ 块结构。

3.3 两种稠密矩阵分解算法的 CUDA Kernel 实现

第四章 面向延迟主元的 GPU 管理方法

第四章部分...

第五章 实验结果与分析

第五章部分...

第六章 总结和展望

第六章部分...

攻读硕士期间发表的论文及其他成果

- [1] 扎克·施耐德, XXX XXX, et al. XXXX Title[C/OL]/III H D, SINGH A. Proceedings of Machine Learning Research: Proceedings of the 37th International Conference on Machine Learning: vol. 119. [S.l.]: PMLR, 2020: XXXX-XXXX. <http://proceedings.mlr.press/XXXX.html>.(一作, CCF-A)

致谢

致谢部分...

北京大学学位论文原创性声明和使用授权说明

原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

论文作者签名： 日期： 年 月 日

学位论文使用授权说明

(必须装订在提交学校图书馆的印刷本)

本人完全了解北京大学关于收集、保存、使用学位论文的规定，即：

- 按照学校要求提交学位论文的印刷本和电子版本；
- 学校有权保存学位论文的印刷本和电子版，并提供目录检索与阅览服务，在校园网上提供服务；
- 学校可以采用影印、缩印、数字化或其它复制手段保存论文；
- 因某种特殊原因需要延迟发布学位论文电子版，授权学校☐一年/☐两年/☐三年以后，在校园网上全文发布。

(保密论文在解密后遵守此规定)

论文作者签名： 导师签名：

日期： 年 月 日

